



DPUInterface

The **DPUInterface** can be used to provide an unified interface for simulator-specific functions.

For this, the **DPUInterface** runs usually on the same computer and in the same thread as the encapsulated simulator-specific DPUs.

In this case **DPUInterface** doesn't cause additional latency and only leads to renaming of variables.

Another application of **DPUInterface** is to create replays of simulations and to play those. The **DPUInterface** optionally provides the opportunity to record, respectively read (out of a recorded file), the values of its variables in every simulation step.

1. In- and Outputs:

<any desired name>

In-/Outputs are defined through parameter VARSet.

2. Static parameters:

VARSet

Defines the variables which shall be provided.

Syntax is *VARSet* = { *Name1*, *Name2*, ... };

All variables stated here can be linked to the implementation DPU. Please also refer to section „Functional principle“.

Interface_Mode

0 (Standard): No recording, no playback

1: Record contents of all variables

2: Record contents of all variables, ignore launch data (SaveDataToken, DataFileNameToken)

3: Play content of all variables

Interface_Postfix

Appendix for file name of the recording file.

Interface_FileName

File name of recording file. (Standard: DPUInterface)

Interface_Extension

File name extension of recording file (Standard: .rec)

Interface_Path

Standard: \SILAB\DATA

Interface_LD_SaveDataToken

Name for launch data, which states if a recording file shall be created. (Only relevant for Mode=1)

Interface_LD_DataFileNameToken

Name for launch data, which states a file name for the recording file. (Only relevant for Mode=1)

Interface_OverwriteExistingFile

Should an existing recording file be overwritten (true) or should numbered files be created instead (false)?

3. Outputs:

Interface_Stop

If a replay is played the *Interface_Stop* is set to true as soon as the end of the recorded data is reached.

Details:



Parameter **VARSet** lists variables, which the interface should provide. The Syntax is as follows:

```
VARSet = {  
    Name1,  
    Name2,  
    ...  
};
```

By default, all these variables are created as floating point numbers. Optionally an-other type can be selected, by writing the type in parentheses after the variable name:

```
VARSet = {  
    Name1(double),  
    Name2(string)  
};
```

Following types are currently supported:

Type name	Description
double	Floating point number
String	Character string, i.e. a text value.

The data type for an interface variable should be same as the type of the DPU variable which is connected.

4. Functional principle:

During the recording of replays (*Interface_Mode=1* oder *Interface_Mode=2*) the file name of the recording file is being built by attaching the values of the following parameters to each other in this order:

Interface_Path, *Interface_FileName*,
Interface_Postfix and *Interface_Extension*

For *Interface_Mode=1* *Interface_FileName* is replaced by the value of the launch data *Interface_LD_DataFileNameToken*, if this launch data is defined.

Additionally a number is suffixed, too, if *Interface_OverwriteExistingFile = false* is set and a recording file with this name already exists.

Data of the recording file are recorded in a SILAB-specific binary format. If data shall be recorded for processing or evaluation it is recommended to use **DPUDataFile** instead of **DPUInterface**.

5. Example:

In SILAB default configuration the DPUInterface is used in many cases for unification of variable names. Example:

```
DPUInterface Mockup  
{
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
Computer = {VDYN};  
Index = 0; # when using DPUInterface Index is not relevant  
Interface_Postfix = "_Mockup";
```

```
VARSet = {  
    Ignition,  
    Starter,  
    Gear,  
    AcceleratorPedal,  
    AcceleratorKickdown,  
    BrakePedal,  
    BrakeReadiness,  
    SteeringWheel,  
    SteeringWheelv,  
    OffsetSteeringMoment,  
    SteeringMoment,  
    SteeringMomentModel,  
    IndicatorLeft,  
    IndicatorRight,  
    HandBrake,  
    Horn,  
    UpperBeam,  
    LowerBeam,  
    FogLight  
};
```

```
Ignition = 1;  
Starter = 1;
```

```
};
```

If a replay of a simulation run should be created, the mockup input can be recorded. Therefore, a separate DPU pool is defined:

```
DPUPool CreateReplay : Full  
{  
    Executable = true;  
    Mockup.Interface_Mode = 2;  
};
```

To play the replay, another DPU pool is created:

```
DPUPool PlayReplay : Full  
{  
    Executable = true;  
    Mockup.Interface_Mode = 3;  
};
```